

Criando a sua Primeira Aplicação Modular

Objetivo da Aula

O principal objetivo desta aula é capacitar o aluno a integrar os conceitos fundamentais do React à construção de componentes reutilizáveis e modulares, além de compreender como as propriedades e os estados da aplicação podem ser utilizados na construção de uma aplicação modular, com base no princípio da responsabilidade única.

Apresentação

Seja bem-vindo à aula de **criação da sua primeira aplicação modular** no React. Nesta etapa do nosso aprendizado, iremos mergulhar na construção de uma aplicação mais complexa e organizada, adotando o conceito de modularização para otimizar o desenvolvimento.

Iniciaremos explorando a estrutura base da aplicação de gerenciamento de tarefas, utilizada como referência de modelo de interface da aplicação.

Na sequência, construiremos uma versão inicial da aplicação, contendo todos os elementos em um único componente principal. Isso nos permitirá compreender os desafios e as limitações dessa abordagem e nos motivará a buscar soluções mais eficientes.

Em seguida, mergulharemos na modularização da aplicação. Entenderemos a importância de dividir nossa aplicação em partes menores e independentes, cada uma com uma responsabilidade única. Essa prática nos permitirá ter um código mais organizado, legível e reutilizável, facilitando a colaboração em equipe e o crescimento da aplicação de forma sustentável.

1. Estrutura Base da Aplicação

Na construção de aplicativos, utilizando o React, é importante pensar na aplicação como um conjunto de componentes, que podem ser montados para a construção da interface. Entretanto, é comum quem está iniciando com a tecnologia pensar no componente como uma página. Nada o impedirá de construir toda a aplicação em um só componente.

Nesta aula, construiremos uma aplicação de gerenciamento de tarefas. Tenha como referência o modelo de interface apresentado na Figura 1.

Figura 1: Interface da aplicação de tarefas

The mockup shows a web application titled "Aplicativo de Tarefas". At the top, there is a dark blue header with the title in white. Below the header, there is a white input field followed by a button labeled "Nova tarefa". The main content area is divided into two sections, both with teal headers. The first section, "Tarefas pendentes:", contains two yellow task cards. Each card has a task name in blue text ("Estudar React" and "Estudar Node") and a button labeled "Tarefa concluída?". The second section, "Tarefas Finalizadas:", contains two yellow task cards with blue text ("Estudar React Native" and "Estudar JavaScript").

Fonte: Elaboração própria.

Na Seção 2, você irá construir a aplicação de gerenciamento de tarefas em um único componente. Em seguida, Na Seção 3, você irá entender a importância de modularizar a interface em pequenos componentes que possuem uma única responsabilidade. Por fim, ao modularizar a interface em pequenos componentes, você identificará os principais desafios no processo, como a propagação de props.

2. Criando a Aplicação em um Único Componente

Primeiramente, crie um projeto React, utilizando o comando `npx create-react-app app-tarefas` (ou crie um projeto utilizando a ferramenta on-line CodeSandbox - <https://codesandbox.io>).

2.1. Definindo a Estrutura da Aplicação

Vamos estruturar os elementos da interface em *src/App.js*, de acordo com o código apresentado a seguir.

```
import './App.css';  
export default function App() {  
  return (  
    <div className="App">  
      <header className='Cabecalho'>  
        <h1>Aplicativo de Tarefas</h1>  
      </header>  
      <div className='NovaTarefa'>  
        <input type='text' placeholder='Digite uma nova tarefa' />  
        <button type='button'>Nova Tarefa</button>  
      </div>  
      <div className='ListaTarefas'>  
        <h2>Tarefas Pendentes:</h2>  
        <div className='Tarefa'>  
          <span>Estudar React</span>  
          <button type='button'>Tarefa concluída?</button>  
        </div>  
        <div className='Tarefa'>  
          <span>Estudar Node</span>  
          <button type='button'>Tarefa concluída?</button>  
        </div>  
      </div>  
      <div className='ListaTarefas'>  
        <h2>Tarefas Concluídas:</h2>
```

```

    <div className='Tarefa'>
      <span>Estudar React Native</span>
    </div>

    <div className='Tarefa'>
      <span>Estudar JavaScript</span>
    </div>
  </div>
</div>
);
}

```

A interface resultante carece de estilização, embora apresente a mesma estrutura mostrada na Figura 1, o que faremos na Seção 2.2.

2.2. Aplicando Estilização aos Elementos da Aplicação

Os elementos declarados em *src/App.js* seguem uma estrutura de contêineres que agrupa os elementos principais de cada parte da interface, cada qual possui uma classe para ser estilizada. A esses contêineres, o layout construído com **flexbox**, *display: flex*, é ideal para produzir interfaces web responsivas a dispositivos de diferentes tamanhos.



Destaque

Flexbox organiza os elementos-filho do contêiner em colunas ou linhas e determina como os elementos-filho serão espaçados, alinhados e/ou ajustados de forma **flexível**, dentro do contêiner.

Em *src/App.css* (ou em *src/styles.css* no CodeSandbox), utilize o trecho de código abaixo para aplicar a estilização às classes *Cabecalho*, *NovaTarefa*, *ListaTarefas* e *Tarefa*, utilizadas pelos contêineres.

```
.Cabecalho {  
    background-color: #282c34;  
    display: flex;  
    font-size: 16px;  
    justify-content: center;  
    color: white;  
}  
  
.NovaTarefa {  
    margin: 10px;  
}  
  
.ListaTarefas {  
    background-color: darkcyan;  
    color: white;  
    padding: 10px;  
    margin: 10px;  
}  
  
.Tarefa {  
    background-color: blanchedalmond;  
    color: blue;  
    display: flex;  
    flex-direction: row;  
    justify-content: space-between;  
    padding: 5px;  
    margin: 5px;  
}
```

Observe que as classes **Cabecalho** e **Tarefa** utilizam **display: flex**. A classe **Cabecalho** centraliza o título da aplicação e preenche os espaços vazios do contêiner, com a cor de

fundo #282c34. Por sua vez, *Tarefa* organiza os elementos em linha, alinhando à esquerda o nome da tarefa, e à direita o botão de conclusão.

2.3. Gerenciando o Estado da Aplicação

Os itens apresentados nas listas de tarefas foram definidos manualmente, e a aplicação carece de gerenciamento de estado para que possamos adicionar novas tarefas e defini-las como concluídas. Para gerenciar o estado das tarefas, utilize o código a seguir no topo da função *App()*.

```
const [tarefas, setTarefas] = useState([
  { id: 1, titulo: 'Estudar React', completa: false },
  { id: 2, titulo: 'Estudar Node', completa: true }
]);
```

Para listar as tarefas pendentes e concluídas, utilizando o estado inicial do componente, você pode utilizar as funções *filter()* e *map()*. O código apresentado a seguir introduz ao primeiro contêiner as tarefas pendentes encontradas na lista e as mapeia em elementos JSX, para exibir na interface o título da tarefa e um botão de conclusão. Ao segundo contêiner, serão apresentadas apenas as tarefas concluídas, decoradas com um risco.

```
<div className='ListaTarefas'>
  <h2>Tarefas Pendentes:</h2>
  {tarefas.filter(tarefa => !tarefa.completa)
    .map(tarefa => (
      <div key={tarefa.id} className='Tarefa'>
        <span>{tarefa.titulo}</span>
        <button type='button'>Tarefa concluída?</button>
      </div>
    ))}
</div>

<div className='ListaTarefas'>
```

```
<h2>Tarefas Concluídas:</h2>
{tarefas.filter(tarefa => tarefa.completa)
  .map(tarefa => (
<div key={tarefa.id} className='Tarefa'>
  <span style={{textDecoration: 'line-through'}}>
    {tarefa.titulo}
  </span>
</div>
))}
</div>
```

É importante notar que, em cada contêiner de lista, os objetos mapeados devem sempre ser identificados unicamente através da propriedade especial `key`. Isso permite ao React identificar mudanças de estado quando alguma operação for realizada na lista.

Para reagir às mudanças da aplicação, vamos criar as funções ***novaTarefa()*** e ***concluirTarefa()***, conforme o código a seguir.

```
function novaTarefa() {
  const input = document.querySelector('div.NovaTarefa input');
  if(input.value === '') return;
  const id = tarefas[tarefas.length-1].id + 1;
  const novaTarefa = { id, titulo: input.value, completa: false };
  setTarefas([...tarefas, novaTarefa]);
  input.value = '';
}

function concluirTarefa(tarefa) {
  const index = tarefas.findIndex(t => t.id === tarefa.id);
  tarefas[index].completa = true;
  setTarefas([...tarefas]);
}
```

A função `novaTarefa()` procura pelo elemento `input`, dentro de um elemento `div`, com a classe `NovaTarefa`. Caso o input seja válido, o valor desse input é adicionado como uma nova tarefa da lista, e o estado é atualizado. Para que essa função seja executada, adicione a função ao evento `onClick` do botão, conforme o seguinte trecho de código: `<button type='button' onClick={novaTarefa}>Nova Tarefa</button>`.

A função `concluirTarefa(tarefa)` é responsável por procurar a tarefa na lista e atualizar seu estado para “concluída”. Para que essa função seja acionada, quando o botão “Tarefa concluída?” for clicado, é necessário adicionar essa função ao evento de clique do botão. Além disso, para passar o parâmetro da tarefa para a função `concluirTarefa`, é necessário usar uma **arrow function** no seguinte trecho: `<button type='button' onClick={() => concluirTarefa(tarefa)}>Tarefa concluída?</button>`.

O código-fonte resultante dessa aplicação está disponível em <https://github.com/daniel-souza/gran-frontend/tree/main/app-tarefas>. (acesso em 19 set. 2023.).



Para Refletir

A construção da aplicação em um único componente se mostrou eficaz para a aplicação de gerenciamento de tarefas. Entretanto, se a sua aplicação precisar atender a mudanças constantes do usuário ou se ela precisa crescer e escalar, como você visualizaria essa abordagem?

3. Modularizando a Aplicação em Pequenos Componentes

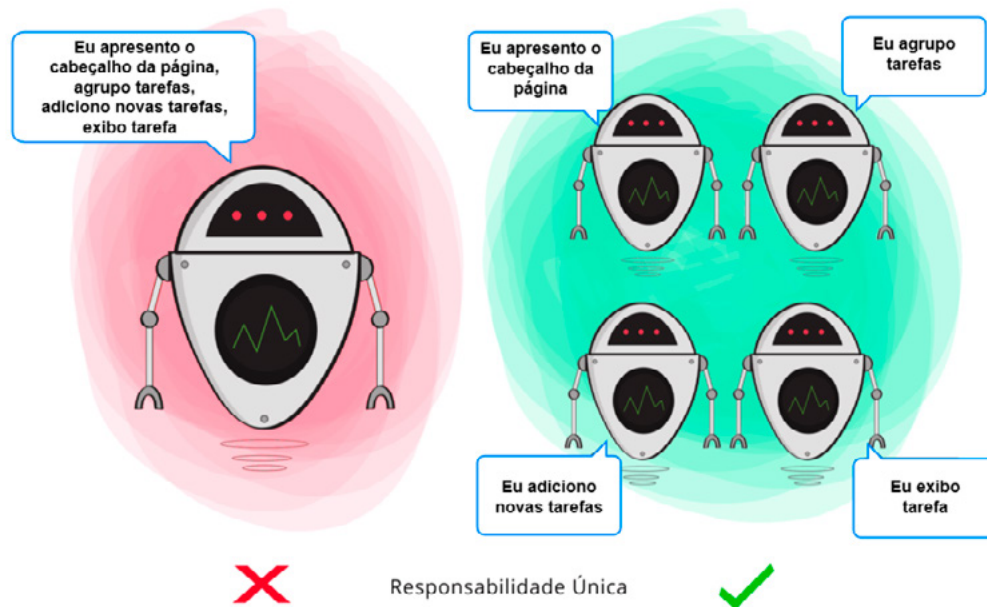
A seguir, realizaremos a modularização da aplicação construída anteriormente, utilizando o princípio da responsabilidade única. Começaremos, primeiro, entendendo como funciona esse princípio; em seguida, vamos à prática.

3.1. O Princípio da Responsabilidade Única

Conforme Minnick (2022), cada componente deve seguir o princípio da responsabilidade única (SRP), do inglês Single Responsibility Principle. O SRP é fundamental no desenvolvimento de componentes React. Esse princípio estabelece que cada componente deve ter apenas uma única responsabilidade, ou seja, deve realizar apenas um tipo de papel. Ao invés de criar um grande componente, que realiza várias funcionalidades ao mesmo tempo, seguindo o SRP você cria componentes menores e mais simples, que são mais fáceis de testar e reutilizar em

diferentes partes da aplicação. Conforme a Figura 2, nossa aplicação pode ser decomposta em quatro componentes, cada qual com sua própria responsabilidade na aplicação.

Figura 2: Ilustração do princípio da responsabilidade única



Fonte: Adaptada de Thelma (2020). Disponível em: <https://medium.com/backticks-tildes/the-s-o-l-i-d-principles-in-pictures-b34ce2f1e898>. Acesso em: 1 jul. 2023.

3.2. Modularizando a Aplicação

Dando continuidade ao projeto anterior, podemos criar um componente chamado **Cabecalho**.

```
export default function Cabecalho() {
  return (
    <div className="Cabecalho">
      <h1>Aplicativo de Tarefas</h1>
    </div>
  )
}
```

Em contrapartida, o componente **NovaTarefa** já não tem acesso direto ao estado da lista de tarefas, e, portanto, o estado e a função de atualização precisam ser propagados via props.

```
export default function NovaTarefa({ tarefas, setTarefas }) {  
  function novaTarefa() {  
    const input = document.querySelector('div.NovaTarefa input');  
    if(input.value === '') return;  
    const id = tarefas[tarefas.length-1].id + 1;  
    const novaTarefa = { id, titulo: input.value, completa: false };  
    setTarefas([...tarefas, novaTarefa]);  
    input.value = '';  
  }  
  return (  
    <div className='NovaTarefa'>  
      <input type='text' placeholder='Digite uma nova tarefa' />  
      <button type='button' onClick={novaTarefa}>Nova Tarefa</button>  
    </div>  
  )  
}
```

Por sua vez, o componente **ListaTarefas** pode ser reutilizado para diferentes propostas. Como em nossa aplicação as tarefas possuem apenas dois estados, podemos reutilizar esse componente para a construção de listas pendentes e concluídas. Essas tarefas são filtradas por estado e mapeadas em componentes **Tarefa**.

```
import Tarefa from "../Tarefa";  
export default function ListaTarefas({ titulo, tarefasFiltradas, ...  
props }) {  
  return (  
    <div className='ListaTarefas'>
```

```

    <h2>{titulo}</h2>

    {tarefasFiltradas.map(tarefa => (
    <Tarefa key={tarefa.id} tarefa={tarefa} {...props} />
    ))}
  </div>
)
}

```

As propriedades do estado da aplicação, *tarefas*, e sua função de atualização, *setTarefas*, são propagadas de *App* → *ListaTarefas* → *Tarefa*, o que poderá mudar o estado de pendente para tarefa concluída. Assim como o componente *ListaTarefas*, o componente *Tarefa* também sofreu alterações, para que possa ser reutilizado quando a tarefa estiver pendente ou concluída.

```

export default function Tarefa( { tarefa, tarefas, setTarefas } ) {
  function concluirTarefa(tarefa) {
    const index = tarefas.findIndex(t => t.id === tarefa.id);
    tarefas[index].completa = true;
    setTarefas([ ...tarefas ]);
  }
  return (
    <div key={tarefa.id} className='Tarefa'>
      <span style={{textDecoration: tarefa.completa && 'line-through'}}>
        {tarefa.titulo}
      </span>
      {!tarefa.completa
        && <button type='button'
          onClick={() => concluirTarefa(tarefa)}>
            Tarefa concluída?
          </button>}
    </div>
  )
}

```

```

</div>
)
}

```

Por fim, o componente **App** ficou mais simples e apenas com a responsabilidade de compor os elementos essenciais para a interface. Observe que o componente **ListaTarefas** pôde ser reutilizado para listar tarefas tanto concluídas quanto pendentes de forma separada.

```

import { useState } from 'react';
import './App.css';
import Cabecalho from './components/Cabecalho';
import NovaTarefa from './components/NovaTarefa';
import ListaTarefas from './components/ListaTarefas';
export default function App() {
  const [tarefas, setTarefas] = useState([
    { id: 1, titulo: 'Estudar React', completa: false },
    { id: 2, titulo: 'Estudar Node', completa: true }
  ]);
  return (
    <div className="App">
      <Cabecalho />
      <NovaTarefa tarefas={tarefas} setTarefas={setTarefas} />
      <ListaTarefas titulo="Tarefas Pendentes:"
        tarefasFiltradas={tarefas.filter(t => !t.completa)}
        tarefas={tarefas} setTarefas={setTarefas} />
      <ListaTarefas titulo="Tarefas Concluídas:"
        tarefasFiltradas={tarefas.filter(t => t.completa)}
        tarefas={tarefas} setTarefas={setTarefas} />
    </div>
  );
}

```

Da mesma forma, caso você decida exibir, em uma única listagem, as tarefas tanto pendentes quanto concluídas, basta passar a lista **tarefas**, sem a etapa de filtragem, à propriedade **tarefasFiltradas**.

O código-fonte resultante dessa aplicação está disponível em <https://github.com/daniel-souza/gran-frontend/tree/main/app-tarefas-srp>. (acesso em 19 set. 2023.).



Destaque

O procedimento de propagação de estado de um componente a outro é uma prática comum, denominada de **lifting**, conforme a documentação oficial do React (React. Dev, 2023).

Considerações Finais da Aula

Ao final desta aula, podemos destacar a importância de compreender e aplicar os princípios de modularização em nossos projetos. Através desse aprendizado, fomos capazes de evoluir de uma aplicação monolítica, em um único componente, para uma estrutura modular mais organizada e eficiente.

Na sequência, ao criarmos a primeira versão da aplicação em um único componente, compreendemos os desafios e as limitações dessa abordagem. A falta de modularização pode levar a um código extenso, difícil de gerenciar e com alta complexidade.

No entanto, ao aplicarmos a modularização da aplicação em pequenos componentes, percebemos a transformação positiva em nosso código. Componentes menores e independentes, cada um com uma responsabilidade única, tornam o desenvolvimento mais ágil, claro e reutilizável. Além disso, facilitam a identificação e correção de problemas, tornando o projeto mais robusto e confiável.

Ao concluir esta aula, esteja ciente da importância de manter um código bem-organizado e modular. A aplicação dos conceitos aprendidos aqui não apenas aprimora a qualidade do código, mas otimiza a produtividade e facilita as futuras implementações e atualizações.

Material Complementar

**Vídeo**

Se Você Passar Por Esses 5 Desafios, Você Aprendeu React JS

2021, Filipe Deschamps.

Filipe Deschamps providenciou um vídeo que impulsionará o seu aprendizado em React. Ele enumera cinco desafios: 1. entender a semelhança do React com tags HTML; 2. criar o primeiro componente React; 3. acessar a propriedade Children de um componente; 4. fazer o componente ter memória (state); e, por fim, 5. entender o comportamento declarativo da parte visual.

Link para acesso: <https://www.youtube.com/watch?v=aJR7f45dBNs>

(acesso em 18 set. 2023.)

Referências

CODESANDBOX. Disponível em: <https://codesandbox.io>. Acesso em: 1 jul. 2023.

MINNICK, Chris. **Beginning ReactJS Foundations Building User Interfaces with React-JS: An Approachable Guide**. ed. 1. Nova Jersey: Wiley, 2022.

REACT. **Sharing State Between Components**. React, 2023. Disponível em: <https://react.dev/learn/sharing-state-between-components>. Acesso em: 1 jul. 2023.

SOUZA, Daniel S. **gran-fronted**. Github, 2023. Disponível em: <https://github.com/daniel-souza/gran-frontend>. Acesso em: 1 jul. 2023.

THELMA, Ugonna. **The S.O.L.I.D Principles in Pictures**. Backticks & Tildes, 18 maio 2020. Disponível em: <https://medium.com/backticks-tildes/the-s-o-l-i-d-principles-in-pictures-b34ce2f1e898>. Acesso em: 1 jul. 2023.